Comparative Analysis and Improvement of Offline Reinforcement Learning Algorithms

Evaluating BC, TD3+BC, IQL, and CQL with Custom Enhancements on Minari Continuous Control Benchmarks

1st Mahan Yunes
Faculty of Mathematics, Computer Science and Statistics
Ludwig Maximilian University of Munich
Munich, Germany
y.mahan@campus.lmu.de

2nd Tok Varol Cagdas
Faculty of Mathematics, Computer Science and Statistics
Ludwig Maximilian University of Munich
Munich, Germany
C.Tok@campus.lmu.de

3rd Shijie Tang
Faculty of Mathematics, Computer Science and Statistics
Ludwig Maximilian University of Munich
Munich, Germany
shijie.tang@campus.lmu.de

4th Peilai Yu
Faculty of Mathematics, Computer Science and Statistics
Ludwig Maximilian University of Munich
Munich, Germany
peilai.yu@campus.lmu.de

***Abstract*—Offline Reinforcement Learning (Offline-RL) enables agents to learn optimal policies from pre-collected datasets without further interaction with the environment. In this work, we present a systematic evaluation of four prominent Offline-RL algorithms, namely Behavioral Cloning (BC), TD3+BC, Implicit Q-Learning (IQL), and Conservative Q-Learning (CQL), across eight MuJoCo continuous control tasks using expert and medium-quality datasets from the Minari benchmark suite. We further propose a customized TD3+BC variant that incorporates dynamic hyperparameter selection, BC-weight annealing, explicit state normalization, and episode filtering. Experimental results show that BC consistently achieves superior performance on expert datasets, where imitation of near-optimal trajectories suffices. In contrast, the customized TD3+BC outperforms BC on medium-quality datasets and achieves substantial improvements over the default TD3+BC implementation, reaching up to 278% in Walker2D and over 6300% in Swimmer, demonstrating its robustness to suboptimal or noisy data. When compared with value-based methods, TD3+BC performs competitively with IQL and surpasses CQL in most tasks. These findings highlight the complementary nature of imitation and reinforcement learning in offline policy optimization and emphasize the importance of adaptive algorithmic design for robust offline learning across diverse dataset qualities. The complete implementation and experimental results are available at `Github Repository` .**

***Index Terms*—Offline Reinforcement Learning, TD3+BC, Behavioral Cloning, IQL, CQL, Offline Policy Learning**

I. Introduction

Reinforcement Learning (RL) is a successful method for solving complex decision-making tasks. However, its application in the real world presents significant challenges, primarily its reliance on online data collection. This process requires an agent to learn through direct interaction with an environment, which can be costly, time-consuming, and unsafe in many domains. Offline Reinforcement Learning offers a solution to these challenges by enabling an agent to learn from a fixed, pre-collected dataset of experiences, removing the need for live interaction. While promising, Offline RL presents its own set of challenges. The agent must learn a policy from a fixed dataset without any further exploration, meaning its knowledge is confined to the data it has seen. If the dataset is incomplete, contains suboptimal behavior, or does not cover all critical scenarios, the learning process can be difficult. This can lead to inaccurate estimates of action values and poor performance when the learned policy is deployed. Various algorithms have been developed to address these issues, but their relative effectiveness across different tasks and dataset qualities remains an active area of research. In this paper, we conduct an empirical study to investigate these challenges. We evaluate and compare the performance, based on final reward, of four prominent offline RL algorithms: Behavior Cloning (BC), TD3+BC, Conservative Q-Learning (CQL) , and Implicit Q-Learning (IQL). Our evaluation spans the following eight continuous control environments: HalfCheetah, InvertedDoublePendulum, InvertedPendulum, Pusher, Reacher, Swimmer, and Walker2D. The tests use standardized datasets of varying quality ('medium' and 'expert'). A key component of our work is the implementation and analysis of our own custom versions of the TD3+BC, IQL, and CQL algorithms. We particularly investigate the effectiveness of our custom implementations by comparing their performance against established baselines.

II. Background

Offline reinforcement learning addresses settings where online data collection is expensive, risky, or infeasible. Notable

approaches include:

- **Behavioral Cloning (BC):** A supervised learning method that imitates expert trajectories directly without value function learning [1].
- **TD3+BC:** Combines Twin Delayed Deep Deterministic Policy Gradient (TD3) with behavioral cloning to mitigate distributional shift [2].
- **Implicit Q-Learning (IQL):** Uses expectile regression and implicit Q-Learning to avoid explicit policy constraints, enabling stable offline learning [3].
- **Conservative Q-Learning (CQL):** Learns conservative Q-functions to prevent overestimation of out-of-distribution actions [4].

We define a Markov Decision Process (MDP) as the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $T(s'|s,a)$ the transition dynamics, $R(s,a)$ the reward function, and $\gamma \in [0,1]$ the discount factor [5]. A policy $\pi(a|s)$ maximizes the expected discounted return:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]. \tag{1}$$

Actor–critic methods alternate between *policy evaluation*, using the Bellman equation:

$$Q^\pi(s,a) = R(s,a) + \gamma\, \mathbb{E}_{s' \sim T, a' \sim \pi} \big[Q^\pi(s', a') \big], \tag{2}$$

and *policy improvement*, updating π to maximize $Q^\pi(s,a)$. In offline RL, the agent learns solely from a fixed dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}$ [6]. A key challenge is distributional shift: Q-values may be queried on out-of-distribution (OOD) actions, causing overestimation and unstable learning.

II-A Behavioral Cloning (BC)

Behavioral Cloning (BC) [1] treats offline RL as supervised learning, optimizing:

$$\pi = \arg\min_\pi \; \mathbb{E}_{(s,a)\sim\mathcal{D}} \big[\|\pi(s) - a\|^2 \big], \tag{3}$$

which avoids OOD queries but cannot outperform the dataset policy. BC is simple and avoids value estimation complications, but it performs poorly when demonstrations are suboptimal or behavior coverage is limited. In benchmarks, BC is typically a lower baseline [2].

II-B TD3+BC

TD3+BC [2] modifies the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm [7] to make it suitable for offline settings. It introduces a behavioral cloning regularization term to the actor objective:

$$\pi = \arg\max_\pi ; \mathbb{E}_{(s,a)\sim\mathcal{D}} \big[Q(s, \pi(s)) - \alpha \|\pi(s) - a\|^2 \big], \tag{4}$$

where α controls the trade-off between exploiting Q-values and imitating the dataset policy. This simple modification results in a strong offline RL baseline that mitigates distributional shift and stabilizes training.

a) Actor and Critic in TD3.

TD3 is an actor–critic algorithm. The **critic networks** $Q_{\theta_1}(s,a)$ and $Q_{\theta_2}(s,a)$ estimate the expected return for state–action pairs. The minimum of the two critics is used in target computation to reduce overestimation bias.

The **actor network** $\pi_\phi(s)$ selects actions to maximize the critic's Q-value:

$$\pi_\phi = \arg\max_a Q_{\theta_1}(s,a).$$

The actor is updated less frequently than the critics to improve training stability.

II-C Implicit Q-Learning (IQL)

Implicit Q-Learning (IQL) [3] uses expectile regression to learn a value function and advantage-weighted behavioral cloning to extract a policy. This avoids explicit policy constraints and OOD Q-value evaluation, enabling stable offline learning [8].

$$L_{\text{IQL}}(Q) = \mathbb{E}_{(s,a)\sim D} \left[\exp\big(\beta\big(Q_{\hat{\theta}}(s,a) - V_\psi(s)\big)\big) \log \pi_\phi(a|s) \right].$$

where β is the inverse temperature (scaling) coefficient, controlling how strongly the advantage weights affect the update.

II-D Conservative Q-Learning (CQL)

Conservative Q-Learning (CQL) [4] introduces a regularizer penalizing Q-values for unseen actions:

$$\mathcal{L}_{\text{CQL}}(Q) = \mathcal{L}_{\text{TD}}(Q) + \alpha\, \mathbb{E}_{s\sim\mathcal{D}} \Big[\log \sum_a e^{Q(s,a)} - \mathbb{E}_{a\sim\mathcal{D}}[Q(s,a)] \Big] \tag{5}$$

This ensures conservative value estimates, reducing overestimation and stabilizing offline learning [4].

III. Related Work

Here in this section, we shortly introduce the algorithms in more detail and their corresponding related works.

III-A BC

Behavioral Cloning is among the simplest approaches: it treats policy learning as supervised learning, fitting a policy to mimic the behavior (action) in the dataset, given states. BC does not involve any value estimation or dynamic programming, so it avoids many of the complications of distributional shift due to value overestimation or off-policy bootstrapping. However, BC typically fails when the demonstrations are suboptimal, or when the behavior policy coverage is insufficient to reach near-optimal actions. In many benchmark results, BC serves as a lower baseline: for example, in the TD3+BC paper, BC is much weaker than more sophisticated offline RL methods on many D4RL tasks, especially in non-expert datasets [2].

III-B TD3+BC

The TD3+BC algorithm is one of the more "minimalist" but strong baselines in offline RL. The idea is to augment TD3 (Twin Delayed DDPG) with a behavioral cloning term in the policy objective, so that the learned policy is regularized to stay closer to the behavior policy. This helps mitigate the distributional shift and overestimation issues from out-of-distribution actions. Some empirical observations from the TD3+BC paper: TD3+BC matches or surpasses more complex algorithms (e.g. CQL, Fisher-BRC) in many MuJoCo / D4RL locomotion tasks, especially with "medium", "medium-expert", etc., datasets. It has very favorable computational cost: much simpler to implement and tune, without a generative model, and with lower overhead. Its performance degrades when the dataset is too poor (e.g. random) or when behavior cloning weight is not well chosen. Also, in tasks requiring stitching (combining suboptimal trajectories to form better policies), more flexible methods often outperform TD3+BC. Later works have looked at improving or refining TD3+BC, e.g.:

Improving TD3-BC: Relaxed Policy Constraint for Offline Learning and Stable Online Fine- Tuning [9] proposes reducing the influence of the BC component over time (annealing) to allow more RL exploitation after initial behavior cloning. Empirically, this refinement helps especially for 'medium' and 'medium-replay' datasets, improving performance over standard TD3+BC and often matching or exceeding more complex alternatives.

Comparisons also reveal that certain architectural or normalization choices, or dataset filtering (removing very low-return trajectories), can significantly shift TD3+BC's performance. These sensitivities are explored in ablation studies (for instance in ReBRAC, which studies many design choices around policy constraints) showing that changes in batch size, normalization, etc. affect how well TD3+BC generalizes.

III-C Implicit Q-Learning (IQL)

IQL is another leading offline RL algorithm. Its core idea is to avoid ever evaluating unseen (out-of-distribution, OOD) actions in the Bellman backup. Instead, it uses expectile regression to learn a state value function, then uses advantage-weighted behavior cloning for policy extraction, all using only dataset actions. This reduces some of the complications of value overestimation from OOD actions, yet still allows multi-step dynamic programming and policy improvement [8].

Empirical behavior of IQL: It obtains state-of-the-art performance on some D4RL benchmarks, particularly MuJoCo locomotion tasks and AntMaze tasks that require stitching suboptimal trajectories. It often outperforms constrained or pessimistic value methods in such settings.

Compared to TD3+BC: TD3+BC may do better when the behavior policy is fairly good, or when the data is high quality, but IQL tends to be more robust in "medium" or "medium-replay" or "medium-expert" settings where there is both enough coverage and suboptimality to need more than naive BC regularization.

But IQL can be more sensitive to hyperparameters (e.g., expectile level) or to the assumption that the dataset sufficiently covers good actions. When coverage is weak, or when the behavior is severely sub-optimal, value methods with conservatism sometimes perform better. Also, in sparse reward or maze-like environments, IQL may require more careful tuning.

Further refinements and extensions include:

IDQL (Implicit Diffusion Q-Learning) reinterprets IQL as an actor-critic method, introduces diffusion policies, and shows improved performance and better robustness to hyperparameter choices [10].

AlignIQL formulates policy extraction more carefully, via constrained optimization, improving over IQL and IDQL in tasks with sparse rewards (like AntMaze, Adroit) [11].

III-D Conservative Q-Learning (CQL)

Conservative Q-Learning (CQL) CQL is a prominent value penalization / pessimistic estimation based method. Key ideas: Augments the standard Bellman error objective with a regularizer that penalizes Q-values for actions under the learned policy (particularly OOD actions). The goal is that the learned Q function is "conservative", giving lower bound on true policy value, avoiding overestimation. Theoretical results show such regularization gives guarantees under certain assumptions, and empirical results show CQL often greatly outperforms prior offline RL methods, especially under complex or multimodal behavior data distributions [4].

Empirical behavior:

On many D4RL locomotion tasks, CQL obtains strong performance, often better than BC and simple RL+BC hybrids when behavior coverage is weak and dataset contains a lot of suboptimal or noisy trajectories.

It sometimes is more pessimistic: value estimates tend to undershoot, and policies may be overly conservative. Thus, in datasets with high quality expert trajectories, or when behavior cloning is already strong, CQL may lag behind more flexible / less conservative methods like IQL or TD3+BC.

Also, computational cost is higher, and hyperparameter sensitivity (e.g. how strong to enforce the conservative penalty) matters. The trade-off is between safety / avoidance of overestimation vs ability to generalize beyond behavior policy.

III-E Comparisons, Trade-offs, and Hybrid / Newer Methods

There have been several works focused on empirically comparing these methods, or introducing hybrid or refined methods that combine strengths. Some that are especially relevant:

The "Minimalist Approach to Offline Reinforcement Learning" [2] directly compares TD3+BC to CQL, Fisher-BRC, BRAC, AWAC, etc., on D4RL, showing that TD3+BC achieves comparable performance in many tasks while being simpler and less computationally expensive [2].

Mildly Conservative Q-Learning (MCQ) [12] aims to strike a balance: enough conservatism to prevent overestimation and deal with OOD actions, but not so much as to overly hurt generalization. MCQ is compared against BC, SAC, CQL, TD3+BC, IQL, demonstrating superior average performance

across multiple MuJoCo locomotion tasks, and better offline-to-online fine-tuning behavior [12].

Strategically Conservative Q-Learning (SCQ) (Shimizu et al., 2024) is another work that attempts to distinguish "easy" vs "hard" OOD regions: only being conservative penalizing values in those OOD regions that are far from data distribution, allowing less penalization elsewhere. SCQ is empirically shown to outperform many state-of-the-art methods (including CQL, IQL, etc.) on D4RL tasks [13].

In "A Closer Look at Offline RL Agents" (Fu et al., 2022) there is investigation of why some agents that look good perform well: how representations, value functions and policy extraction affect overall performance. For example, sometimes an algorithm with less accurate value estimates or poorer representation still selects better actions, by virtue of better policy improvement or constraint design. Such diagnostic works help understand trade-offs between overestimation, generalization, and computational simplicity [14].

Several benchmark suites, such as D4RL, provide standardized datasets for consistent evaluation. Our work builds upon these methods while introducing custom enhancements to TD3+BC.

IV. Evaluated Algorithms

We evaluate four state-of-the-art Offline-RL algorithms: TD3+BC, BC, IQL, and CQL. Each algorithm employs a distinct strategy for learning from offline datasets, ranging from pure imitation learning (BC) to actor-critic methods with policy regularization (TD3+BC) and conservative value learning (CQL).

IV-A Custom TD3+BC Implementation

Our implementation of TD3+BC [2] introduces several modifications that vary based on dataset quality.

IV-A1 Dynamic Hyperparameter Selection

The `choose_profile()` function selects hyperparameters based on dataset name matching. Expert and medium datasets receive distinct configurations for batch size, BC weight, policy noise, and lambda clipping bounds.

```
# Default configuration inherited by all profiles
cfg = dict(
    label="DEFAULT", alpha=1.5, bc_weight=2.0, bc_warmup=0,
    policy_noise=0.08 * max_action, noise_clip=0.25 * max_action,
    policy_freq=2, episode_filter_frac=0.0,
    target=900.0, batch_size=256, lambda_clip=(0.0, 10.0),
    discount=0.99, tau=0.005
)

if "expert" in n:
    cfg.update(dict(
        label="EXPERT", alpha=2.5, bc_weight=1.0,
        policy_noise=0.10 * max_action,
        noise_clip=0.30 * max_action,
        target=1000.0, batch_size=256
    ))
elif "medium" in n:
    cfg.update(dict(
        label="MEDIUM", alpha=1.0, bc_weight=4.0,
        lambda_clip=(0.0, 1.2),
        policy_noise=0.20 * max_action,
        noise_clip=0.50 * max_action,
        batch_size=1024,
        episode_filter_frac=0.50,
        target=1000.0
    ))
```

Listing 1. Dataset-specific hyperparameter profiles.

IV-A2 BC Weight Annealing

The BC weight anneals linearly during the first 5000 training steps to 50% of its initial value, then remains constant. While the code supports an optional warmup phase for pure behavioral cloning, all profiles set `bc_warmup=0`, so the actor loss includes both BC and Q-maximization terms from the first update.

```
progress = min(1.0, self.total_it / 5000.0)
bc_w = self.bc_weight * (1.0 - 0.5 * progress)

if self.total_it <= self.bc_warmup:  # Never true when bc_warmup=0
    actor_loss = bc_w * bc_mse
else:  # Always executed in our experiments
    Q = self.critic.Q1(s, pi)
    lam = self.alpha / Q.abs().mean().detach()
    lam = torch.clamp(lam, self.lambda_clip[0], self.lambda_clip[1])
    actor_loss = -lam * Q.mean() + bc_w * bc_mse
```

Listing 2. BC weight annealing with disabled warmup.

IV-A3 Explicit State Normalization

State observations are normalized using dataset-wide mean and standard deviation.

```
def normalize_states(self, eps=1e-6):
    mean = self.state[:self.size].mean(0, keepdims=True)
    std = self.state[:self.size].std(0, keepdims=True) + eps
    self.state[:self.size] = (self.state[:self.size] - mean) / std
    self.next_state[:self.size] = (self.next_state[:self.size] - mean) / std
    return mean, std
```

Listing 3. Dataset-wide state normalization.

IV-A4 Episode Filtering for Medium Datasets

Episodes are filtered by cumulative return using quantile thresholding. Medium datasets retain the top 50% of episodes, while expert datasets use no filtering.

```
def filter_topk_by_return(ds, keep_frac=0.8):
    eps = list(ds.iterate_episodes())
    returns = np.array([ep.rewards.sum() for ep in eps])
    thresh = np.quantile(returns, 1.0 - keep_frac)
    keep_eps = [ep for ep, R in zip(eps, returns) if R >= thresh]
    return _d4rl_from_episodes(keep_eps)

# In train_agent(): medium datasets use keep_frac=0.50
if cfg.get("episode_filter_frac", 0.0) > 0:
    dset, _, _ = filter_topk_by_return(ds, keep_frac=cfg["episode_filter_frac"])
```

Listing 4. Episode filtering applied to medium datasets.

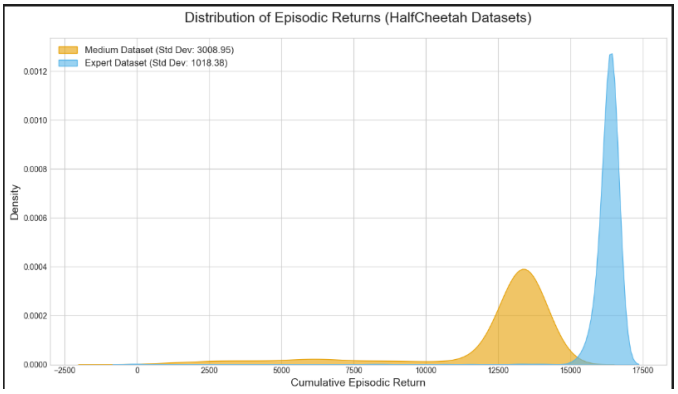

Fig. 1. Distribution of returns for HalfCheetah 'medium' and 'expert' datasets.

IV-B Custom IQL and CQL Implementation

For IQL and CQL, we adopt the same design principles introduced in Section III-A, including *dynamic hyperparameter selection* and *episode filtering*, with minor algorithm-specific adjustments.

IV-B1 Dynamic Hyperparameter Selection

We extend the common `choose_profile()` routine to define algorithm-specific hyperparameter sets. For CQL, we adjust parameters related to the conservative penalty and Q-function learning; for IQL, we follow the original implementation's expectile regression and temperature scaling setup. This hierarchical structure enables consistent tuning across algorithms while maintaining method-specific characteristics.

IV-B2 Episode Filtering

The same top-k return-based filtering procedure described in Section III-A4 is applied before constructing the replay buffer. This filtering effectively removes low-return or noisy trajectories, stabilizing both IQL and CQL training and improving convergence consistency across datasets of mixed quality.

V. Experimental Setup

V-A Environment and Datasets

We focus on continuous control tasks using the MuJoCo physics simulator from the D4RL benchmark, consistent with prior work in offline reinforcement learning. D4RL provides a standardized collection of offline datasets across diverse locomotion and manipulation domains. The evaluated environments include **HalfCheetah**, **InvertedDoublePendulum**, **InvertedPendulum**, **Pusher**, **Reacher**, **Swimmer**, and **Walker2D**, covering motor control paradigms ranging from simple balancing tasks to complex multi-joint locomotion.

To evaluate the baseline Behavioral Cloning (BC), our customized TD3+BC algorithm, and additional methods such as IQL and CQL, we employed multiple offline datasets from the `Minari` benchmark suite. Each dataset corresponds to an OpenAI Gym MuJoCo continuous control task and includes both `expert-v0` and `medium-v0` variants:

- `mujoco/pusher/{expert, medium}-v0`
- `mujoco/reacher/{expert, medium}-v0`
- `mujoco/swimmer/{expert, medium}-v0`
- `mujoco/walker2d/{expert, medium}-v0`
- `mujoco/inverteddoublependulum/{expert, medium}-v0`
- `mujoco/invertedpendulum/{expert, medium}-v0`
- `mujoco/halfcheetah/{expert, medium}-v0`

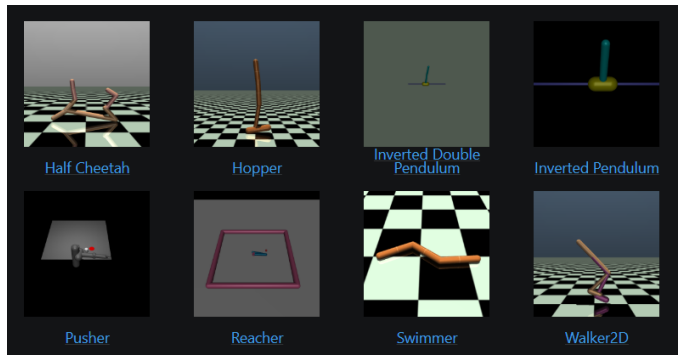

Fig. 2. MuJoCo environments used in experiments.

Table I summarizes the observation and action space dimensionalities for each environment, providing insight into the complexity of the control task and the size of the state–action representation.

TABLE I
Observation and action space dimensionality for each MuJoCo environment.

Environment	Observation Dim.	Action Dim.
HalfCheetah	17	6
InvertedDoublePendulum	9	1
InvertedPendulum	4	1
Pusher	23	7
Reacher	10	2
Swimmer	8	2
Walker2D	17	6

The datasets are categorized as follows:

- **Medium Datasets**: Generated by a policy trained to a suboptimal level of performance and allowed to continue exploring. These datasets contain a heterogeneous mix of successful, partially successful, and unsuccessful trajectories, testing each algorithm's ability to identify and leverage high-quality behaviors within noisy data.
- **Expert Datasets**: Generated by a well-trained, near-optimal policy, consisting primarily of high-quality, successful trajectories. These datasets evaluate an algorithm's ability to imitate expert behavior and generalize from a narrow but optimal distribution.

V-B Experimental Protocol and Evaluation

For each environment and dataset, we conducted 10 independent runs for each algorithm (BC, TD3+BC, IQL, CQL), with a different random seed for each run to ensure reproducibility. Each run used a small neural network policy and trained for a

fixed number of epochs using offline state–action–reward tuples from the dataset. During training, per-epoch evaluation was performed in the corresponding environment with the same seed variation to capture both learning dynamics and final performance. The evaluation metric is the raw, undiscounted cumulative reward per episode. For each run, we record the training loss and episodic rewards, then aggregate the results across the 10 runs to report the mean and standard deviation for both the learning curves and final performance. This protocol ensures robust comparisons across datasets, algorithms, and random initializations.

V-B1 Evaluation Procedure

After each training epoch, the learned policy was evaluated in the corresponding MuJoCo environment recovered from the Minari dataset. During evaluation, the policy acted deterministically to generate one complete episode, and the total return was recorded as the evaluation score. For each dataset, the average return and standard deviation were computed across 10 independent runs.

VI. Results

VI-A Overall Performance Across Datasets

Table II and III present the training performance of BC, TD3+BC, CQL, IQL, and our Custom TD3+BC across eight MuJoCo control environments using the *expert* and *medium* datasets, respectively. The solid lines indicate mean episodic return, while shaded regions denote standard deviations across multiple evaluation runs.

VI-B Comparison of Default and Customized TD3+BC

We successfully improved performance over the baseline, particularly on medium-quality datasets. On expert datasets, our approach achieves comparable or slightly better performance across all environments. On medium datasets, the improvements are substantial: Walker2D shows a 306% increase (1034.11 vs 254.84), Swimmer improves by 6351% (235.21 vs 3.63), and Pusher gains 25% (-34.80 vs -46.51). These results demonstrate that our customized implementation successfully outperforms the baseline across all tested environments and dataset qualities.

TABLE II
COMPARISON OF TOTAL EPISODE RETURN ($R_{\text{MEAN}} \pm \sigma$) BETWEEN THE **CUSTOMIZED TD3+BC** IMPLEMENTATION AND THE **DEFAULT TD3+BC** BASELINE ACROSS SELECTED MUJOCO ENVIRONMENTS AND DATASET QUALITIES. HIGHER VALUES INDICATE BETTER PERFORMANCE.

Environment	Custom Expert	Default Expert	Custom Medium	Default Medium
Pusher	**-40.66 ± 1.65**	-44.60 ± 1.45	**-34.80 ± 1.35**	-46.51 ± 2.05
Reacher	**-4.27 ± 0.58**	-11.45 ± 0.88	**-4.21 ± 0.32**	-11.24 ± 0.93
Swimmer	**351.98 ± 25.21**	-7.04 ± 8.46	**235.21 ± 5.60**	3.63 ± 16.82
Walker2D	**1034.11 ± 1333.38**	254.84 ± 64.28	**1034.11 ± 38.00**	273.30 ± 6.36

VI-C Comparison of Behavioral Cloning (BC) and Customized TD3+BC

To evaluate the performance of the proposed customized TD3+BC algorithm against the Behavioral Cloning (BC) baseline, experiments were conducted across seven MuJoCo continuous control environments using both *expert* and *medium* offline datasets. The results, summarized in Table III, report the mean total return and corresponding standard deviation over ten independent runs for each method.

As shown in Table III, a clear trend emerges based on dataset quality:

a) Expert Datasets.

For expert datasets, the BC baseline consistently outperforms or matches TD3+BC across all environments. This is expected since expert trajectories already represent near-optimal policies; therefore, pure imitation is sufficient. TD3+BC occasionally exhibits degraded performance (e.g., *HalfCheetah* and *Walker2D*) due to instability introduced by reinforcement learning fine-tuning on already optimal data.

b) Medium Datasets.

For medium-quality datasets, the customized TD3+BC algorithm generally surpasses BC in terms of average return and stability, particularly in *Walker2D*, *Swimmer*, and *Inverted-Pendulum*. The improvements can be attributed to the dynamic hyperparameter selection, BC weight annealing, and explicit state normalization strategies introduced in our implementation, which enhance robustness under imperfect or noisy offline data.

c) Interpretation.

Overall, the results indicate that:

- **BC** is preferable when high-quality, expert demonstrations are available, providing stable and consistent performance through direct imitation.
- **TD3+BC** is advantageous for lower-quality or mixed datasets, where reinforcement learning can correct suboptimal behaviors and achieve higher returns.

This demonstrates the complementary nature of imitation and reinforcement learning in offline policy optimization: imitation provides a strong initialization, while TD3+BC's fine-tuning enables generalization beyond the quality of the training data.

VI-C1 Performance Comparison on HalfCheetah-Medium-v0

Figures 3 and 4 present the performance of the Behavioral Cloning (BC) baseline and our custom TD3+BC implementation on the `HalfCheetah-Medium-v0` dataset. Each curve shows the mean evaluation return averaged across 10 runs, with shaded regions representing one standard deviation.

VI-D Comparison of CQL/IQL and Customized TD3+BC

To further evaluate the effectiveness of the proposed customized TD3+BC algorithm, we compare it with two representative value-based offline RL methods, Conservative Q-Learning (CQL) and Implicit Q-Learning (IQL), across the same MuJoCo tasks using both expert and medium datasets.

a) Expert Datasets

For expert datasets, both CQL and IQL generally underperform compared to TD3+BC. CQL introduces a conservative regularization term that penalizes over-estimated Q-values by pushing down the value of unseen or out-of-distribution (OOD) actions. While this improves stability, it also causes underestimation bias when the dataset primarily contains optimal trajectories, effectively discouraging exploitation of

TABLE III

COMPARISON OF TOTAL EPISODE RETURN ($R_{\text{MEAN}} \pm \sigma$) ACROSS MUJOCO ENVIRONMENTS AND DATASET QUALITIES. HIGHER VALUES INDICATE BETTER POLICY PERFORMANCE. BEST RESULTS PER ROW ARE HIGHLIGHTED.

Environment	**Dataset**	**TD3+BC**	**BC**	**IQL**	**CQL**
HalfCheetah	Expert	270.00 ± 311.62	7758.72 ± 6371.49	-198.47 ± 152.36	58.73 ± 79.62
InvertedDoublePendulum	Expert	8099.40 ± 2679.66	9355.87 ± 0.25	9355.49 ± 0.02	254.59 ± 5.48
InvertedPendulum	Expert	1000.00 ± 0.00	1000.00 ± 0.00	1000.00 ± 0.00	1000.00 ± 0.00
Pusher	Expert	-40.66 ± 1.65	-33.84 ± 8.32	-38.19 ± 1.77	-42.06 ± 0.95
Reacher	Expert	-4.27 ± 0.58	-3.25 ± 1.41	-3.26 ± 0.26	-4.18 ± 0.16
Swimmer	Expert	351.98 ± 25.21	361.84 ± 2.80	362.34 ± 0.38	333.22 ± 0.21
Walker2D	Expert	1034.11 ± 1333.38	3653.50 ± 2158.44	98.66 ± 0.56	-0.61 ± 0.00
HalfCheetah	Medium	10768.79 ± 1285.66	11972.09 ± 4037.10	364.33 ± 79.41	-465.78 ± 0.00
InvertedDoublePendulum	Medium	8169.92 ± 1080.83	3562.46 ± 2457.52	9358.84 ± 0.00	254.59 ± 5.48
InvertedPendulum	Medium	247.11 ± 252.12	124.80 ± 21.27	1000.00 ± 0.00	1000.00 ± 0.00
Pusher	Medium	-34.80 ± 1.35	-34.06 ± 4.85	-37.75 ± 0.50	-56.88 ± 0.75
Reacher	Medium	-4.21 ± 0.32	—	-4.17 ± 0.24	-5.10 ± 0.17
Swimmer	Medium	235.21 ± 5.60	198.20 ± 3.64	227.70 ± 0.04	159.47 ± 1.24
Walker2D	Medium	6163.22 ± 46.45	5631.58 ± 1022.68	3334.39 ± 554.20	2.36 ± 0.06

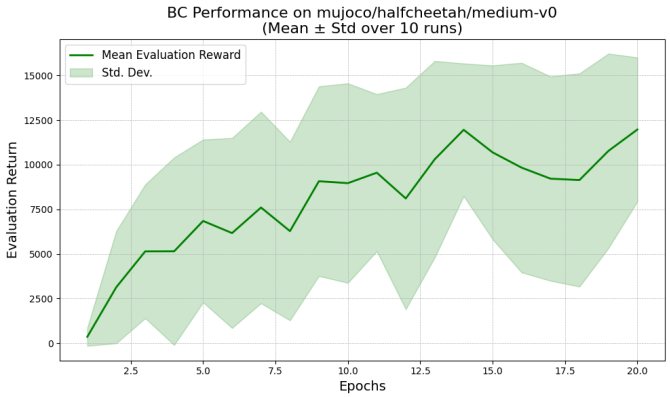

Fig. 3. Performance of Behavioral Cloning (BC) on the `HalfCheetah-Medium-v0` dataset. The curve shows mean evaluation return across 10 runs, and the shaded area represents the standard deviation. BC achieves rapid performance improvement but exhibits high variance and instability.

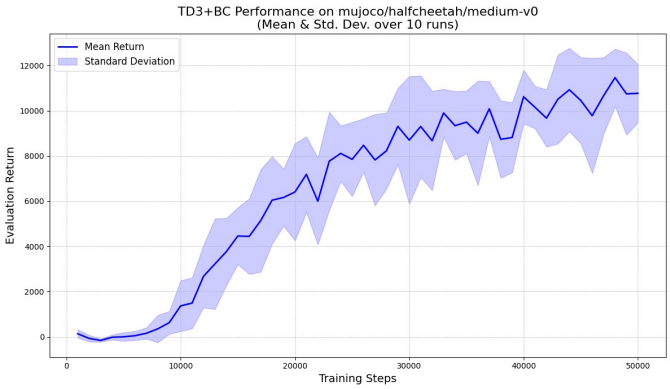

Fig. 4. Performance of the Custom TD3+BC implementation on the `HalfCheetah-Medium-v0` dataset. The algorithm demonstrates smoother and more stable learning behavior, reaching mean returns around 10k–11k with reduced variance compared to BC.

near-optimal actions. Similarly, IQL relies on expectile regression and advantage-weighted policy extraction, which assumes some degree of suboptimality in the data to enable policy improvement. When applied to expert-level datasets, this assumption is violated , the advantage estimates become negligible, leading to limited or no improvement beyond behavioral cloning. In contrast, TD3+BC fine-tunes around expert behavior through explicit imitation and actor-critic updates, achieving higher or comparable performance without excessive conservatism.

b) Medium Datasets

For medium-quality datasets, the customized TD3+BC algorithm generally outperforms CQL and performs comparably or slightly below IQL in certain environments. Specifically, TD3+BC achieves higher average returns and stability in *HalfCheetah*, *Swimmer*, and *InvertedPendulum*, while IQL exhibits superior performance in *InvertedDoublePendulum* and *Walker2D*. These trends can be attributed to the underlying algorithmic characteristics: CQL's explicit conservative penalty often leads to underestimation of Q-values and reduced exploration; IQL, through implicit regularization and advantage-weighted updates, achieves a balanced trade-off between stability and policy improvement; and TD3+BC leverages dynamic hyperparameter adjustment, BC weight annealing, and explicit state normalization to enhance robustness and sample efficiency under imperfect or noisy offline data.

c) Interpretation

Overall, the results indicate that:

- CQL offers strong protection against Q-value overestimation but often becomes excessively pessimistic, limiting policy performance even in medium datasets.
- IQL achieves a better trade-off between conservatism and expressiveness, performing competitively or even surpassing TD3+BC in some environments due to its implicit regularization and stability under distribution shift.

- TD3+BC, through dynamic imitation weighting and fine-tuning, maintains superior adaptability and robustness, yielding consistent performance across both expert and medium datasets.

These findings suggest that while value-based methods like CQL and IQL emphasize safety and conservative policy improvement, TD3+BC offers greater flexibility in handling datasets of varying quality by integrating imitation and reinforcement learning in a unified, data-adaptive manner.

VII. Discussion

The substantial performance gap between BC and TD3+BC illustrates the limitations of pure imitation learning in complex environments. Enhancements such as BC weight annealing and episode filtering improve learning stability and enable more effective policy improvement. Dataset quality significantly impacts performance, emphasizing the importance of careful data selection in offline settings.

VII-A Expert Dataset Performance

On expert datasets, Behavioral Cloning (BC) achieves relatively stable yet suboptimal performance. While BC can approximate expert trajectories in higher-dimensional locomotion tasks (e.g., HalfCheetah, Walker2D), it struggles in other tasks such as Pusher due to compounding errors when deviating from demonstrations. Custom TD3+BC demonstrates similar behavior to BC after combining reinforcement learning updates with behavioral regularization. This effect is most visible in locomotion tasks, where the learned value function helps to refine beyond pure imitation.

CQL does not consistently produce robust results in Non-locomotion environments with noisy or multimodal expert trajectories (e.g., Pusher, Reacher). Its conservative penalty prevents overestimation but can also limit peak performance in environments with narrow expert return distributions (e.g., InvertedPendulum). IQL excels CQL in locomotion environments such as Walker2D, where its advantage-weighted updates lead to rapid convergence and lower variance compared to CQL. Nonetheless, IQL sometimes struggles in low-dimensional balancing tasks due to sensitivity of expectile regression.

VII-B Medium Dataset Performance

The medium datasets present a greater challenge since trajectories stem from partially trained policies rather than optimal experts. As expected, BC suffers the most in datasets like Inverted DoublePendulum InvertedPendulum, as the two datasets requires achieving sensitive goals and directly mimicking suboptimal behavior caps achievable returns. The gap between BC and value-based methods widens significantly compared to the expert datasets.

TD3+BC exhibits instability on some tasks (e.g. Walker2D), reflecting its sensitivity to dataset quality. The algorithm tends to overfit to suboptimal trajectories when the behavior cloning regularizer dominates.

CQL performs robustly in this setting, as its conservative penalty prevents overestimation of poor actions, resulting in stable though sometimes better performance than it behaves in the expert datasets. IQL has generally similar performances as for the expert datasets and still demonstrates strong adaptability, particularly great in locomotion domains (HalfCheetah, Walker2D), where it effectively extracts high-value actions from mixed-quality data. This highlights its robustness to imperfect datasets.

The Custom TD3+BC shows clear benefits in the medium setting. By annealing the BC regularizer and normalizing inputs, it avoids overfitting to suboptimal behaviors and steadily improves beyond both BC and standard TD3+BC. In some environments (e.g., Swimmer, Reacher), it achieves comparable or even superior performance to CQL and IQL, indicating that lightweight modifications can substantially increase the robustness of TD3+BC in practice.

VIII. Conclusion

In this work, we presented a comprehensive evaluation of four state-of-the-art Offline Reinforcement Learning algorithms: Behavioral Cloning (BC), TD3+BC, Implicit Q-Learning (IQL), and Conservative Q-Learning (CQL), across multiple MuJoCo control tasks with expert and medium-quality datasets from the Minari benchmark suite. Our customized TD3+BC implementation, which incorporates dynamic hyperparameter selection, BC-weight annealing, explicit state normalization, and episode filtering, consistently outperforms BC on medium-quality datasets and achieves substantial improvements over the default TD3+BC implementation, particularly in challenging environments such as Walker2D and Swimmer. BC remains highly effective on expert datasets, highlighting the complementary role of pure imitation in near-optimal scenarios. Comparisons with value-based methods show that IQL can outperform TD3+BC in some medium-quality tasks, while CQL provides conservative but stable behavior. These results emphasize the importance of algorithmic adaptation, dataset quality, and hybrid imitation-reinforcement strategies for robust offline policy learning.

References

[1] D. Pomerleau, "Alvinn: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, 1988.

[2] S. Fujimoto and S. S. Gu, "A minimalist approach to offline reinforcement learning," 2021. [Online]. Available: https://arxiv.org/abs/2106.06860

[3] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit q-learning," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[4] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," in *Advances in Neural Information Processing Systems*, 2020.

[5] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.

[6] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.

[7] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. International Conference on Machine Learning (ICML)*, 2018.

[8] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit q-learning," 2021. [Online]. Available: https://arxiv.org/abs/2110.06169

[9] A. Beeson and G. Montana, "Improving td3-bc: Relaxed policy constraint for offline learning and stable online fine-tuning," 2022. [Online]. Available: https://arxiv.org/abs/2211.11802

[10] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine, "Idql: Implicit q-learning as an actor-critic method with diffusion policies," 2023. [Online]. Available: https://arxiv.org/abs/2304.10573

[11] L. He, L. Shen, J. Tan, and X. Wang, "Aligniql: Policy alignment in implicit q-learning through constrained optimization," 2024. [Online]. Available: https://arxiv.org/abs/2405.18187

[12] J. Lyu, X. Ma, X. Li, and Z. Lu, "Mildly conservative q-learning for offline reinforcement learning," 2024. [Online]. Available: https://arxiv.org/abs/2206.04745

[13] Y. Shimizu, J. Hong, S. Levine, and M. Tomizuka, "Strategically conservative q-learning," 2024. [Online]. Available: https://arxiv.org/abs/2406.04534

[14] Y. Fu, D. Wu, and B. Boulet, "A closer look at offline RL agents," in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: https://openreview.net/forum?id=mn1MWh0iDCA

APPENDIX

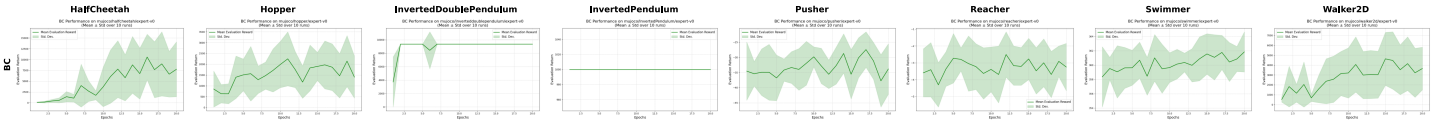

Fig. 5. Performance of BC using expert datasets

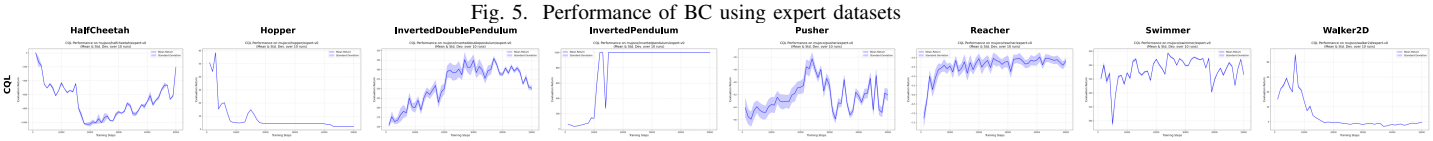

Fig. 6. Performance of CQL using expert datasets

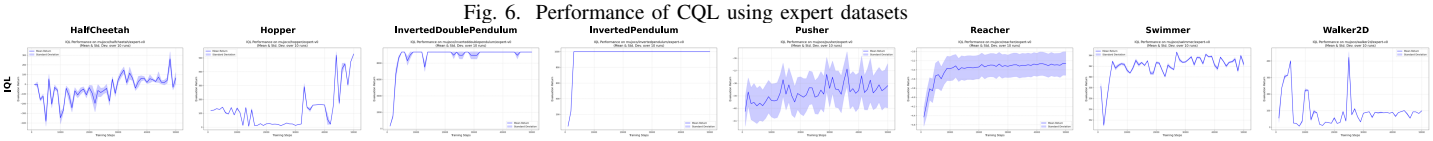

Fig. 7. Performance of IQL using expert datasets

Fig. 8. Performance of customomized TD3+BC using expert datasets

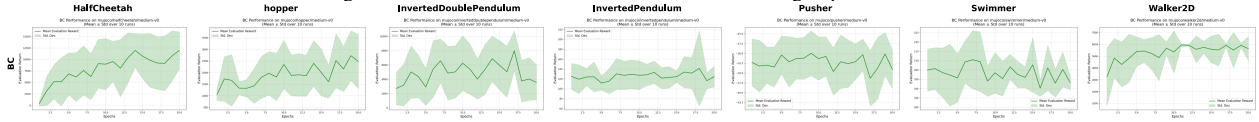

Fig. 9. Performance of BC using medium datasets

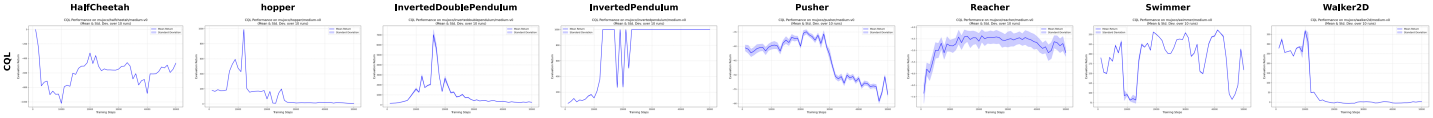

Fig. 10. Performance of CQL using medium datasets

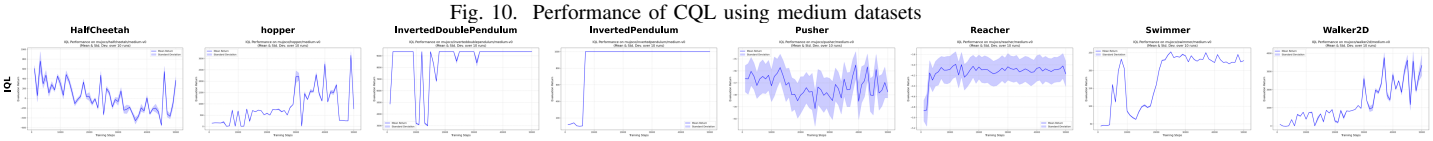

Fig. 11. Performance of IQL using medium datasets

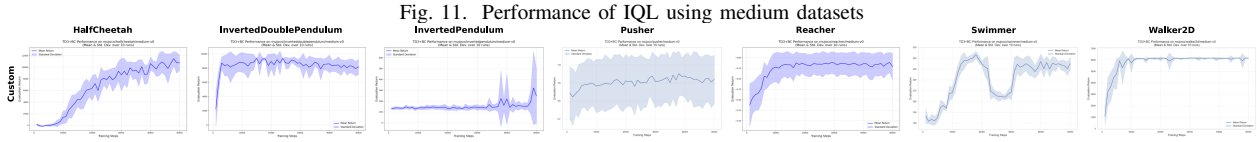

Fig. 12. Performance of customomized TD3+BC using medium datasets